

PipelineInsights — Granular Step-by-Step Build Guide

For: MSBA student, coding/ML beginner **Purpose:** A reference you can return to across sessions. Each phase is broken into small, checkable steps. Check items off as you go — don't move to the next step until the current one works.

How to Use This Document

- Work top to bottom. Don't skip steps even if they seem trivial — beginners save the most time by not skipping "boring" setup steps.
 - Each step has a **Goal**, **What to do**, and **How you'll know it worked**.
 - If a step fails, stop and fix it before moving on. Bugs compound if you build on top of broken foundations.
 - Come back to this doc every session and pick up where your checklist left off.
-

PHASE 1: TENNIS FOUNDATION (Weeks 1-8)

Step 1.1 — Set Up Your Development Environment

Goal: Have a working Python environment where you can write and run code. **What to do:**

1. Install Python (3.10 or higher) if not already installed.
2. Install a code editor — VS Code is the standard recommendation for beginners.
3. Create a project folder, e.g. `pipelineinsights/`.
4. Set up a virtual environment inside it (`python -m venv venv`) so your packages don't conflict with other projects.
5. Activate the virtual environment and install core packages: `pandas`, `numpy`, `scikit-learn`, `xgboost` or `lightgbm`, `requests`, `sqlalchemy`. **How you'll know it worked:** You can run `python -c "import pandas; print(pandas.__version__)"` and see a version number with no errors.

Step 1.2 — Set Up Your Database

Goal: Have a place to store your data (a database, not just CSV files) so it can grow and be queried. **What to do:**

1. Install PostgreSQL (or use SQLite for a lighter-weight start if you want less setup).
2. Create a new database, e.g. `pipelineinsights_db`.

3. Connect to it once from Python using `sqlalchemy` to confirm the connection works.
How you'll know it worked: You can run a simple `SELECT 1;` query from Python and get a result back with no connection errors.

Step 1.3 — Design Your Core Tables

Goal: Have your schema (table structure) mapped out on paper/diagram before writing any code. **What to do:**

1. Sketch these tables (column names, data types): `players`, `matches`, `odds`, `player_prop_predictions`.
2. Keep it simple at first — you can always add columns later. Don't over-engineer.
3. Write the actual `CREATE TABLE` SQL statements and run them against your database.
How you'll know it worked: You can query each empty table (e.g., `SELECT * FROM players;`) and get back an empty result with no errors (proves the table exists with the right structure).

Step 1.4 — Pull Your First Real Data

Goal: Get real tennis match history into your `matches` and `players` tables. **What to do:**

1. Download Jeff Sackmann's ATP (or WTA) match CSV files from his GitHub repo.
2. Write a small Python script that reads one CSV file with `pandas.read_csv()`.
3. Print the first 5 rows (`df.head()`) to see what columns you actually have.
4. Map those columns into your `players` and `matches` tables, and write a script that inserts the data into your database. **How you'll know it worked:** You can query your database and see real player names and match results — not empty tables anymore.

Step 1.5 — Pull Odds Data (Basic Version)

Goal: Get at least some historical or current odds data connected to your matches. **What to do:**

1. Sign up for a free tier of an odds API (e.g., The Odds API).
2. Write a script that calls the API and prints the raw JSON response — just to see the shape of the data.
3. Parse the JSON into rows that match your `odds` table structure.
4. Insert into your database. **How you'll know it worked:** Your `odds` table has real rows linked to real match IDs.

Step 1.6 — Build Your First Feature: Rolling Average

Goal: Calculate a player's average performance over their last N matches — your very first "feature." **What to do:**

1. In Python, group your match data by player, sorted by date.

2. Use `.rolling(window=5).mean()` in pandas to compute a 5-match rolling average for a stat (e.g., total games won).
3. Print the result for one player and sanity-check it by hand — does the number make sense? **How you'll know it worked:** You can look at a real player's rolling average and confirm by eyeballing their last 5 matches that the number is correct.

Step 1.7 — Expand to a Full Feature Table

Goal: Build the `player_game_features` table — the single table all your future models will read from. **What to do:**

1. Add more rolling windows (3, 5, 10 matches).
2. Add head-to-head history (how has this player done against this specific opponent before?).
3. Add rest days (days since their last match).
4. Add surface type (clay/grass/hard) as a column.
5. Write all of this into a new table, `player_game_features`, keyed by player + match ID. **How you'll know it worked:** You can query `player_game_features` for any player-match combination and see a full row of sensible numbers, with no missing/null values for matches that have enough history.

Step 1.8 — Build the Baseline Model

Goal: Your first prediction model — pure rolling average, adjusted for context. **What to do:**

1. Write a Python function: `baseline_prediction = avg_last_5 * context_adjustment`.
2. Start with a simple adjustment (e.g., surface-based multiplier) — don't overcomplicate this yet.
3. Run this function across your whole feature table to generate a prediction for every player-match row.
4. Insert results into `player_prop_predictions` with `modelversion = 'baseline_v1_points'` (or whatever stat you're predicting). **How you'll know it worked:** Your predictions table has one row per player-match with a `baseline_v1` prediction that looks plausible compared to the player's real history.

Step 1.9 — Build the GBM Model

Goal: Train your first real machine learning model. **What to do:**

1. Split your feature table into training data (older matches) and test data (newer matches) — **never shuffle randomly**, always split by date.
2. Pick your target column (e.g., total points/games won next match).
3. Train an XGBoost or LightGBM regressor: `model.fit(X_train, y_train)`.
4. Predict on your test set: `model.predict(X_test)`.

5. Compare predictions vs. actual results — calculate Mean Absolute Error (MAE). **How you'll know it worked:** You get a MAE number that is lower (better) than your baseline model's MAE on the same test set.

Step 1.10 — Add Confidence Intervals via Quantile Regression

Goal: Instead of one prediction number, get a range (low estimate, high estimate). **What to do:**

1. Train two more GBM models with quantile loss objectives — one for the 20th percentile, one for the 80th percentile.
2. Use these as your `lowerci` and `upperci`.
3. Store all three (mean, lower, upper) in `player_prop_predictions` with `modelversion = 'gbm_v1_points'`. **How you'll know it worked:** For most matches, the actual outcome falls between your `lowerci` and `upperci` roughly 60% of the time (since you used 20th/80th percentiles).

Step 1.11 — Build the Ensemble (Weighted Average)

Goal: Combine baseline + GBM into one smarter prediction. **What to do:**

1. Calculate each model's historical MAE (how good has it been).
2. Set weights inversely proportional to error (better model = higher weight).
3. Write: `ensemble_prediction = w_baseline * baseline_pred + w_gbm * gbm_pred`.
4. Store result with `modelversion = 'ensemble_v1_points'`. **How you'll know it worked:** The ensemble's MAE on your test set is lower than either baseline or GBM alone.

Step 1.12 — Build a Reusable Backtesting Script

Goal: A script you can re-run anytime to evaluate any model version against historical data.

What to do:

1. Write a function that takes a date range, pulls predictions + actual results, and computes MAE/RMSE.
2. Make it generic — it should work for any `modelversion` value without hardcoding logic.
3. Save results to a simple CSV or table so you can track model performance over time.

How you'll know it worked: You can run the same script against `baseline_v1`, `gbm_v1`, and `ensemble_v1` and get three different, comparable MAE scores.

✓ **Phase 1 Milestone:** You have real tennis data, a feature table, three working model versions, and a backtesting script that proves the ensemble beats the baseline.

PHASE 2: INTENSIVE TESTING (Weeks 9-14)

Step 2.1 — Run Multi-Season Backtests

What to do: Re-run your backtesting script across at least 2-3 full tennis seasons of historical data, not just one small window. **How you'll know it worked:** You have a table/chart of MAE per model per season, showing consistent (not fluky) performance.

Step 2.2 — Tune Ensemble Weights

What to do: Try a few different weight combinations (e.g., 30/70, 50/50, 70/30 baseline/GBM) and re-run backtests for each. **How you'll know it worked:** You can identify which weight combination minimizes MAE, and you've written down why.

Step 2.3 — Test the Bayesian Model (Optional Layer)

What to do:

1. Identify your lowest-data players (fewest historical matches).
2. Build a simple hierarchical model (e.g., using `PyMC` or `bambi` in Python) with player-level random effects.
3. Compare its accuracy specifically on low-data players vs. your GBM model on the same players. **How you'll know it worked:** You have a clear yes/no answer — did the Bayesian model meaningfully improve predictions for low-data players? If not, document that and move on without building it further.

Step 2.4 — Stress-Test Edge Cases

What to do: Manually pull out matches involving retirements, surface transitions, and debut players. Check how your models performed on these specifically. **How you'll know it worked:** You have a written list of "failure modes" — situations where your model predictably struggles. This becomes your checklist for later sports.

✓ **Phase 2 Milestone:** A validated, backtested tennis model with documented accuracy and a list of known weaknesses.

PHASE 3: PORT TO MLB (Weeks 15-24)

Step 3.1 — Add Team & Schedule Tables

What to do: Create `teams` and `games` tables (games replaces "matches" — now two teams play, not two individuals). Design generically: home/away, date, rest days — so this same structure works for NFL later. **How you'll know it worked:** You can query a full MLB schedule with team names and dates.

Step 3.2 — Pull MLB Data

What to do: Use `pybaseball` (free Python package) to pull Statcast/game log data. Start with one season first. **How you'll know it worked:** You have real MLB player game logs in your database.

Step 3.3 — Rebuild Feature Table for Team Context

What to do: Extend `player_game_features` with team-level features: opponent strength, park factors, pitcher-batter matchup stats. **How you'll know it worked:** Your feature table has both player-level and team-context columns populated for real games.

Step 3.4 — Add Injury/Availability Features

What to do: Find a free injury report source, build an `injuries` table, and add a feature like "is a key teammate out today." **How you'll know it worked:** You can look up a specific game and see whether the injury flag correctly reflects a real reported absence.

Step 3.5 — Reuse Model Code

What to do: Point your existing `baseline.py`, `gbm_model.py`, and `ensemble.py` scripts at the new MLB feature table. Change only the input feature list, not the model logic itself. **How you'll know it worked:** The same code runs without errors and produces MLB predictions.

Step 3.6 — Backtest MLB Model

What to do: Run your reusable backtesting script (from Step 1.12) against MLB predictions. **How you'll know it worked:** You get MAE scores for MLB, comparable in structure to your tennis results.

✓ **Phase 3 Milestone:** A working MLB model, and a written "diff" document listing exactly what had to change to go from tennis to MLB.

PHASE 4: GENERALIZE THE ARCHITECTURE (Weeks 25-30)

Step 4.1 — Extract Feature Lists into Config Files

What to do: Instead of hardcoding which features to use in your Python scripts, create a config file (JSON or YAML) per sport listing the features to use. **How you'll know it worked:** You can switch which sport's model runs by changing a config file, not the code.

Step 4.2 — Abstract the Model Scripts

What to do: Refactor `baseline.py`, `gbm_model.py`, and `ensemble.py` so they accept a feature table and config as inputs, with no sport-specific logic inside them. **How you'll know it worked:** Running the same script on tennis data vs. MLB data only requires changing the input arguments, not the script itself.

Step 4.3 — Build a Generic Reporting View

What to do: Write a SQL view template (`vw_fact_player_prop_odds`) parameterized to work for any sport's tables. **How you'll know it worked:** You can query the view for tennis rows and MLB rows using the same view definition (or near-identical ones).

Step 4.4 — Write Your Own Onboarding Doc

What to do: Write a short internal doc: "To add a new sport: 1) add data source, 2) build ETL, 3) define feature config, 4) run existing model scripts, 5) validate with backtester." **How you'll know it worked:** You could hand this doc to someone else (or future-you) and they could follow it without asking you questions.

✓ **Phase 4 Milestone:** A sport-agnostic framework proven on two sports, with written onboarding instructions.

PHASE 5: NFL BUILD (Weeks 31-42)

Step 5.1 — Pull NFL Data

What to do: Use `nfl_data_py` (wraps nflverse/nflfastR data) to pull play-by-play and player game logs. **How you'll know it worked:** Real NFL player stats are in your database.

Step 5.2 — Define NFL Feature Config

What to do: Using your Phase 4 config system, define NFL-specific features: weather, injury report severity, positional matchups (WR vs. CB). **How you'll know it worked:** Your feature table populates correctly using the new config, without touching your model code.

Step 5.3 — Train & Tune Models

What to do: Run your existing baseline/GBM/ensemble scripts on NFL data. Expect to spend extra time tuning hyperparameters since NFL seasons are short (less data per team). **How you'll know it worked:** Your ensemble model beats the baseline on NFL backtests, even if by a smaller margin than tennis/MLB.

Step 5.4 — Full Historical Backtest

What to do: Backtest across multiple NFL seasons, and compare predictions against real sportsbook lines to calculate hypothetical edge/ROI. **How you'll know it worked:** You have a full performance report for the NFL model, comparable in format to your tennis and MLB reports.

✓ **Phase 5 Milestone:** A working, backtested NFL model built on the same reused architecture.

PHASE 6: DASHBOARD, LIVE TESTING, BUFFER (Weeks 43-54)

Step 6.1 — Build/Extend the Dashboard

What to do: Add a sport-switcher to your Dash app so it can display tennis, MLB, and NFL predictions from the same reporting view. **How you'll know it worked:** You can toggle between sports in the dashboard UI and see correct, sport-specific data for each.

Step 6.2 — Run a Live Pipeline Test

What to do: Let the full pipeline run automatically (via a scheduler like APScheduler) for a real week of NFL games, end to end — ETL, features, models, predictions, dashboard. **How you'll know it worked:** Fresh predictions appear in your dashboard each day without you manually running scripts.

Step 6.3 — Fix What Breaks

What to do: Live pipelines always surface issues backtests don't catch (API rate limits, missing data fields, schema mismatches). Keep a running bug log and fix issues as they appear. **How you'll know it worked:** Your pipeline runs for at least a full week without manual intervention.

✓ **Final Milestone (Sept 1, 2027):** A live, working NFL prop prediction model, built on a proven, reusable architecture validated first on tennis and MLB.